Nama :Veva Br tariagan

NIM :253303621280

Kelas :3 Pagi B

Jurusan :Sistem Infomasi

Dosen :Dr.Evta Indra,S.Kom,.M.Kom

Java Visual

1. Jelaskan perbedaan program sekuensial (Main console di TP1–TP5) dan event-driven (GUI) dengan analogi sehari-hari

   Jawaban:

   Perbedaan program sekuensial dan event-driven

   Program sekuensial menjalankan perintah secara berurutan dari atas ke bawah. Sementara itu, event-driven menjalankan tindakan berdasarkan kejadian atau event, seperti tombol yang diklik pengguna.

   Analogi sehari-hari:

   - Sekuensial: seperti mengikuti resep memasak dari langkah pertama sampai terakhir secara berurutan.
   - Event-driven: seperti mesin ATM yang menunggu pengguna menekan tombol, memilih menu, atau memasukkan PIN sebelum melakukan tindakan.

   Dalam Java, program console pada TP1–TP5 umumnya berjalan berurutan, sedangkan GUI Swing merespons interaksi pengguna melalui event, misalnya ActionListener ketika tombol diklik.
2. Mengapa GUI Swing harus dijalankan lewat SwingUtilities.invokeLater()? Apa risiko jika diabaikan?

   Jawaban:

   SwingUtilities.invokeLater() digunakan untuk menjalankan pembuatan dan pembaruan komponen GUI pada Event Dispatch Thread (EDT), yaitu thread khusus yang menangani event dan tampilan Swing.

   Contohnya:

```
import javax.swing.SwingUtilities;

public class Main {
   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> {
         MainWindow window = new MainWindow(
            new model.CharacterManager()
         );
         window.setVisible(true);
      });
   }
}
```

Jika diabaikan, GUI mungkin tetap muncul, tetapi ada risiko masalah thread-safety, tampilan tidak diperbarui dengan benar, atau perilaku program menjadi tidak konsisten. Karena itu, pembuatan dan pembaruan komponen Swing sebaiknya dilakukan pada EDT.

3. Kode ini membuat jendela tidak muncul: JFrame frame = new JFrame("Test"); JPanel panel = new JPanel(); panel.add(new JLabel("Halo")); frame.add(panel); frame.setSize(400, 300); Identifikasi kesalahannya dan perbaikannya
   Jawaban:
   Kesalahan pada kode JFrame
   Kode awal:

```
JFrame frame = new JFrame("Test");
JPanel panel = new JPanel();
panel.add(new JLabel("Halo"));
frame.add(panel);
frame.setSize(400, 300);
```

   Kesalahannya: belum ada pemanggilan setVisible(true), sehingga jendela belum ditampilkan. Sebaiknya juga tentukan operasi ketika jendela ditutup dan tampilkan setelah seluruh komponen selesai ditambahkan.
   Perbaikannya:

```
import javax.swing.*;

public class Main {
   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> {
         JFrame frame = new JFrame("Test");
         JPanel panel = new JPanel();

         panel.add(new JLabel("Halo"));
         frame.add(panel);

         frame.setSize(400, 300);
         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
         frame.setLocationRelativeTo(null);
         frame.setVisible(true);
      });
   }
}
```

Dengan setVisible(true), jendela akan muncul di layar.

4. Mengapa MainWindow extends JFrame adalah relasi *is-a* sedangkan CharacterManager dengan ArrayList adalah *has-a*? Apa konsekuensinya?
   Jawaban:
   Perbedaan relasi is-a dan has-a
   Relasi is-a berarti suatu class merupakan jenis dari class lain.
   Contohnya:

   ```
   public class MainWindow extends JFrame {
   }
   ```

   MainWindow adalah sebuah JFrame yang memiliki kemampuan dasar jendela Swing, seperti mengatur ukuran dan menampilkan tampilan.
   Relasi has-a berarti suatu class memiliki atau menyimpan objek lain sebagai bagian dari tanggung jawabnya.
   Contohnya:

   ```
   private ArrayList<Character> characters = new ArrayList<>();
   ```

   CharacterManager memiliki daftar objek Character yang dikelolanya.
   Konsekuensinya:

- Is-a: MainWindow mewarisi fitur dan perilaku dari JFrame, tetapi juga terikat pada hierarki pewarisan tersebut.
- Has-a: CharacterManager dapat mengelola daftar karakter tanpa harus menjadi ArrayList. Implementasinya juga lebih mudah diubah tanpa mengubah identitas class tersebut.

5. Di mana proses "ambil data dari CharacterManager lalu tampilkan ke JLabel" sebaiknya diletakkan: di View atau menunggu Controller? Jelaskan dengan prinsip MVC
   Jawaban:
   Di mana proses mengambil data dan menampilkan JLabel sebaiknya diletakkan?
   Dalam prinsip MVC (Model–View–Controller), pembagian tugasnya adalah:
   - Model: menyimpan dan mengelola data karakter, misalnya Character dan CharacterManager.
   - View: menampilkan data ke antarmuka, misalnya menggunakan JLabel di MainWindow.
   - Controller: mengatur alur tindakan pengguna, seperti menerima klik tombol lalu meminta Model memperbarui data dan menyuruh View memperbarui tampilan.

   Jadi, proses menampilkan data ke JLabel sebaiknya berada di View, sedangkan logika pengambilan dan pengelolaan data berada di Model. Controller dapat menjadi penghubung antara keduanya. Untuk aplikasi sederhana yang belum memiliki Controller terpisah, View boleh membaca data dari CharacterManager dan menampilkannya, tetapi sebaiknya tidak mengambil alih logika bisnis Model.